

Rapport de projet Big Data

1. Sujet et objectif

Notre projet porte sur la rentabilisation des Yellow Cabs de New York. Le point de départ, c'est le contexte de concurrence avec les VTC comme Uber. L'idée n'était pas juste de stocker des données, mais d'essayer d'en sortir des informations utiles pour la prise de décisions.

On a donc construit un pipeline pour analyser les trajets de taxis selon plusieurs angles :

- les zones de prise en charge,
- les horaires,
- le nombre de courses et les modes de paiement.

L'objectif final était d'identifier des pistes simples d'amélioration de la rentabilité.

Pour cela, on s'est appuyés sur les données officielles du NYC TLC, qui est l'organisme public en charge des taxis à New York. Ce choix nous semblait pertinent parce qu'on travaille sur des données réelles, volumineuses, et directement liées au sujet. Dans le projet, on exploite les fichiers Yellow Taxi de l'année 2024 (la plus récente disponible) et le référentiel des zones concernées.

2. Logique générale de l'architecture

On a organisé le projet avec une logique de pipeline en trois couches :

- une couche raw/bronze pour ingérer et stocker les données brutes ;
- une couche silver pour nettoyer, contrôler et enrichir les données ;
- une couche gold pour produire des datamarts directement exploitables par une API et un dashboard.

L'ensemble repose sur Hadoop, Spark, Hive, SQLite, FastAPI et Streamlit.

On a utilisé Docker pour lancer l'infrastructure localement. Ce choix nous a permis de simuler un environnement Big Data sans avoir besoin d'un cluster réel dans le cloud. Le ``docker-compose.yml`` lance notamment HDFS, YARN, Spark, Hive et PostgreSQL pour le metastore Hive. En pratique, ça donne un environnement cohérent pour créer une chaîne de traitement de bout en bout.

3. Déroulement du pipeline

3.1. Conversion des fichiers sources

Pendant ce projet j'ai découvert que peu importe si les fichiers sont compressés le .parquet ne changent pas de nom. Les fichiers Parquet d'origine étaient compressés avec ZSTD, alors que notre environnement Spark 3.0 ne les lisait pas correctement.

plutôt que de modifier toute l'infrastructure pour qu'elle puisse lire les fichiers ZSTD, on a décidé de les convertir en fichiers Snappy.

Le script `convertisseur_zstd.py` relit chaque fichier Parquet source avec pandas puis le réécrit en version Snappy. On conserve les fichiers d'origine et on crée de nouveaux fichiers suffixés par `_snappy.parquet`. Ce choix nous a permis de rendre les données compatibles sans changer le reste du pipeline.

3.2. Ingestion dans HDFS avec `feeder.py`

Le script `feeder.py` correspond à la couche bronze. Il lit :

- les fichiers de trajets Yellow Cab au format Parquet ;
- le fichier CSV `taxi_zone_lookup.csv` qui contient le référentiel des zones.

Le script extrait ensuite `year`, `month` et `day` à partir de `tpcp_pickup_datetime`. Ces colonnes servent au partitionnement des trajets pour l'écriture dans HDFS. Ce choix est important, parce qu'un partitionnement par date est plus logique pour des données temporelles de trajets. Cela permet aussi de limiter les volumes lus plus tard si on veut cibler une période précise.

Les trajets sont écrits dans `hdfs://namenode:9000/raw/trips` avec partitionnement par date, et les zones dans `hdfs://namenode:9000/raw/zones`.

On a aussi ajouté deux points d'optimisation dans ce script :

- un `cache()` sur le dataframe partitionné avant un `count()`, pour éviter de recalculer toute la chaîne de transformations juste pour compter les lignes ;
- un `repartition(4)` avant l'écriture, pour mieux contrôler la taille et le nombre de partitions au moment du write.

Ce deuxième point vient d'un problème rencontré pendant le projet : sans repartitionnement, l'écriture provoquait des erreurs mémoire dans notre environnement Docker. On a du forcer 4 partitions pour rendre le traitement plus stable et plus adapté aux ressources limitées du cluster local.

ça on l'a compris grâce au `feeder.txt` qui enregistre les logs de l'exécution et les erreurs.

3.3. Nettoyage et enrichissement avec ``processor.py``

Le script ``processor.py`` constitue la couche silver. Il lit les données brutes depuis HDFS puis applique les règles de qualité principales :

- ``fare_amount > 0`` pour éliminer les courses avec un tarif nul ou négatif
- ``trip_distance > 0`` pour éliminer les courses avec une distance nulle ou négative
- ``tpep_dropoff_datetime > tpep_pickup_datetime`` pour éliminer les courses avec un dropoff antérieur au pickup
- ``passenger_count`` entre 1 et 6 pour éliminer les courses avec un nombre de passagers incohérent
- validité des ``PULocationID`` et ``DOLocationID`` par rapport au référentiel des zones pour éliminer les courses avec un lieu de prise en charge ou de dépose invalide

Ces règles ont été choisies pour éliminer les lignes aberrantes ou inutilisables pour une analyse métier. Par exemple, une course avec un tarif nul ou une distance nulle apporte peu de valeur pour étudier la rentabilité. De la même manière, un ``dropoff`` antérieur au ``pickup`` ou un nombre de passagers incohérent signale une anomalie.

Pour contrôler les zones, on récupère d'abord les ``LocationID`` valides depuis le référentiel, puis on filtre les trajets qui ne correspondent pas à ces identifiants. Ce contrôle est important parce qu'il évite d'introduire des valeurs incohérentes dans les jointures.

Le dataframe est ensuite enrichi avec :

- une jointure sur les zones de départ ;
- une jointure sur les zones d'arrivée ;
- une jointure sur un dictionnaire des types de paiement.

Pour les paiements, on n'a pas utilisé de fichier externe. Le dictionnaire est directement créé dans le code Spark. Comme la table de correspondance est très petite et stable, il était plus pratique de l'intégrer directement dans le script plutôt que d'ajouter une source supplémentaire à maintenir.

Pendant cette phase, on a rencontré un problème de doublon de colonne. La double jointure sur le référentiel des zones dupliquait la colonne ``service_zone``, ce qui bloquait l'écriture ensuite. La solution retenue a été de supprimer cette colonne avant les jointures concernées avec ``.drop("service_zone")``.

Après nettoyage et normalisation, on ajoute aussi deux colonnes utiles :

- ``pickup_hour`` pour travailler sur les heures de départ ;
- ``pickup_date`` pour les agrégations journalières.

Le dataframe enrichi est ensuite mis en mémoire avec ``persist()``. Cette décision a du sens parce que ce même dataframe est réutilisé plusieurs fois derrière pour produire plusieurs sorties. Sans ``persist()``, Spark aurait pu recalculer les jointures et les filtres à chaque nouvelle action, ce qui aurait coûté du temps de traitement.

Le script produit ensuite deux agrégations intermédiaires :

- le revenu moyen par zone de départ et par heure ;
- le nombre de courses par jour et par borough (zone de départ).

Sur la première agrégation, on applique une window function avec ``RANK()``. Le but est de classer les créneaux horaires les plus rentables à l'intérieur de chaque zone de départ. C'est plus riche qu'un simple ``groupBy``, parce qu'on ne veut pas seulement agréger, on veut aussi ordonner les résultats dans chaque groupe.

Les résultats silver sont ensuite écrits dans Hive sous forme de tables :

- ``silver.trips_cleaned``
- ``silver.revenue_par_zone_heure``
- ``silver.courses_par_jour_borough``

Le choix de Hive est cohérent ici, parce qu'on veut stocker la couche silver dans HDFS tout en gardant un accès SQL pour la suite du projet.

Comme pour le feeder, un fichier de log dédié (``processor.txt``) permet de tracer l'exécution.

3.4. Création des datamarts avec ``datamart.py``

Le script ``datamart.py`` correspond à la couche gold. Il relit la table ``silver.trips_cleaned`` depuis Hive avec Spark SQL, puis calcule trois datamarts métier :

- ``dm_zone_performance``
- ``dm_hourly_demand``
- ``dm_payment_analysis``

Le premier sert à analyser les zones de départ les plus rentables, avec le nombre de courses, le tarif moyen et le revenu total. Le deuxième résume la demande par heure. Le troisième permet de comparer les moyens de paiement en volume et en revenu

Une fois ces agrégats calculés dans Spark, ils sont exportés dans une base SQLite locale appelée ``datamarts.db``.

Après agrégation, les données sont beaucoup plus petites que les données de départ. Il n'était donc pas nécessaire d'ajouter une base serveur plus lourde que SQLite juste pour exposer quelques tables de synthèse. Il simplifie énormément le déploiement de l'API, puisque tout repose sur un fichier local.

Là aussi, le script génère son propre fichier de log : ``datamart.txt``.

4. Exposition des résultats

4.1. API FastAPI

L'API a été développée avec FastAPI dans `api/app.py`.

On a choisi FastAPI plutôt que Flask pour plusieurs raisons simples :

- la structure est rapide à mettre en place ;
- la validation des paramètres est propre ;
- c'est très adapté à une petite API de données.

L'API propose :

- un endpoint d'authentification `POST /auth/token` ;
- trois endpoints de lecture sur les datamarts ;
- une pagination avec `limit` et `skip`.

La base SQLite est ouverte à chaque requête via une dépendance FastAPI, ce qui garde le code lisible et évite de laisser des connexions ouvertes inutilement.

L'authentification est gérée par JWT. On a choisi ce mécanisme parce qu'il permet une authentification "stateless" : le serveur n'a pas besoin de stocker une session côté backend. À chaque requête, le token est simplement vérifié. Dans notre code, le token expire au bout d'une heure, ce qui reste raisonnable pour un usage de démonstration.

L'API s'appuie aussi sur un fichier `.env` pour la sécurisation des identifiants. C'est plus propre que de laisser ces valeurs en dur dans le code.

Enfin, toutes les tentatives de connexion et les accès principaux sont loggés dans `app_logs.txt`, ce qui facilite le suivi et le diagnostic.

4.2. Dashboard Streamlit

La dernière partie du projet est un dashboard Streamlit dans ``api/app_streamlit.py``.

On a choisi Streamlit parce que l'objectif n'était pas de développer un front-end complexe, mais de proposer rapidement une visualisation claire des résultats. Streamlit est bien adapté à ce besoin : tout reste en Python, l'intégration avec ``requests``, ``pandas`` et ``plotly`` est directe, et l'interface peut être montée rapidement.

Le dashboard commence par une authentification dans la barre latérale. L'utilisateur saisit son login et son mot de passe, Streamlit appelle l'API pour récupérer un JWT, puis le conserve dans ``st.session_state``.

Une fois connecté, l'utilisateur peut consulter quatre vues :

- l'affluence des taxis par heure ; pour les moments de forte affluence
- la répartition des paiements ; pour les moyens de paiement les plus utilisés
- le top 10 des zones les plus rentables ; pour les zones les plus rentables
- le prix moyen d'une course selon l'heure de départ. pour les heures ou les courses sont les plus chères

Le choix de Plotly se justifie bien ici, parce que les graphiques sont interactifs et restent simples à produire.

5. Justification des choix techniques

Hadoop / HDFS

HDFS était logique pour la couche brute, parce qu'on travaille dans un contexte Big Data et qu'on voulait stocker les données dans un système distribué, nativement compatible avec Spark et Hive. Même en environnement local, cela permet de reproduire une architecture proche de ce qu'on attend dans un vrai pipeline de données.

Spark

Spark était le bon choix pour lire, filtrer, joindre et agréger des volumes de données importants. Un traitement purement en Python aurait été beaucoup moins adapté, surtout pour enchaîner ingestion, nettoyage et calculs analytiques. Le fait de rester dans le même moteur entre bronze, silver et gold simplifie aussi l'architecture.

Hive

Hive apporte une couche SQL sur des données stockées dans HDFS. Pour nous, c'était utile à partir du moment où la couche silver devait être réutilisable et lisible facilement. Cela prépare aussi naturellement le passage vers les datamarts.

SQLite

SQLite n'est pas une technologie Big Data, mais ce n'était plus le besoin à ce stade. Une fois les données agrégées, on voulait juste une base légère, simple à ouvrir depuis l'API, sans serveur supplémentaire. Pour une maintenance ou une démo locale. Mais pour une utilisation plus réaliste, il serait préférable d'utiliser une base de données plus robuste comme PostgreSQL ou MySQL.

FastAPI

FastAPI nous a fait gagner du temps pour exposer les données proprement. La génération automatique de `/docs` est aussi un vrai avantage pour tester l'API pendant le développement.

Streamlit

Streamlit permet de livrer rapidement et simplement une visualisation. C'était un bon compromis entre simplicité de développement et lisibilité du résultat final.

Contrairement à PowerBI ou Tableau, il est gratuit et open source et il est possible de le push sur un dépôt github ce qui a simplifié le travail d'équipe.

6. Optimisations et problèmes rencontrés

Pendant le projet, on a dû traiter plusieurs problèmes concrets.

Le premier concernait la compatibilité de format. Les fichiers sources en ZSTD posaient problème avec Spark 3.0. La conversion en Snappy a permis de sécuriser l'ingestion sans modifier la structure des données.

Le deuxième concernait la mémoire lors de l'écriture des données brutes. Dans notre environnement Docker limité, Spark pouvait être instable. L'ajout de ``repartition(4)`` dans ``feeder.py`` a amélioré la stabilité du traitement.

Le troisième point concernait la réutilisation du dataframe enrichi dans ``processor.py``. Comme ce dataframe servait à plusieurs actions, ``persist()`` a évité des recalculs inutiles.

Le quatrième problème venait du schéma après les jointures sur les zones. La colonne ``service_zone`` apparaissait en double, ce qui bloquait l'écriture. On a corrigé cela en supprimant la colonne avant l'étape problématique.

Enfin, la pagination dans l'API est aussi une petite optimisation fonctionnelle. Même si les datamarts restent petits, il est plus propre de ne pas tout renvoyer d'un coup.

7. Ce que le projet apporte

Le projet permet de passer d'un ensemble de fichiers bruts à une visualisation facilement analysable.

Au lieu de manipuler des millions de lignes de trajets, on obtient à la fin :

- des indicateurs par zone ;
- une vision de la demande par heure ;
- une analyse des modes de paiement ;
- le top 10 des zones les plus rentables ;
- le prix moyen d'une course selon l'heure de départ.

On a essayé d'aller jusqu'à une vraie mise à disposition des résultats pour les utilisateurs finaux.

8. Limites du projet

Le projet reste un prototype, donc il a aussi plusieurs limites.

D'abord, l'authentification repose sur un seul couple login / mot de passe et une base SQLite locale. Ce n'est pas fait pour de la production.

Ensuite, certaines décisions sont volontairement simples, par exemple le dictionnaire des paiements codé en dur ou l'absence d'orchestration avancée du pipeline.

Enfin, le cluster Docker local permet de simuler l'architecture, mais il ne correspond pas totalement aux performances d'une vraie infrastructure distribuée.

9. Conclusion

Ce projet nous a permis de construire une chaîne Big Data complète, depuis les fichiers sources jusqu'à la visualisation finale. Le point important, selon moi, c'est qu'on n'a pas seulement empilé des technologies : chaque choix avait une logique par rapport au besoin.

HDFS et Spark servaient à absorber et traiter les données volumineuses. Hive structurait la couche silver. SQLite simplifiait la mise à disposition des datamarts. FastAPI et Streamlit permettaient ensuite de rendre les résultats visibles et utilisables.

Au final, le projet répond bien à la problématique de départ : transformer des données brutes de trajets en indicateurs concrets pour mieux comprendre où et quand les Yellow Cabs sont les plus rentables.

Encore merci pour le temps en plus qui nous a été accordé. Grâce à ça j'ai pu vraiment comprendre le fonctionnement de Spark et de son écosystème et je me suis rendu compte que j'en avais besoin même si le cours était clair!